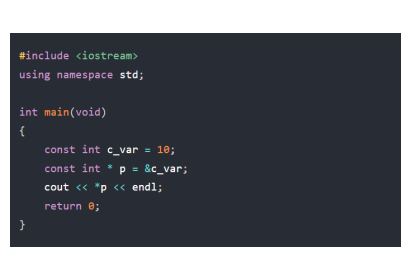
WHITE ALUBMN

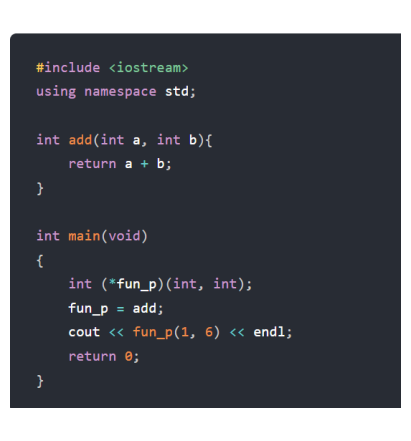
C

- 指针体系（一级/二级指针、指针与数组、指针与字符串）
  - 指针基础
    - 什么是指针？
      - 指向另外一种类型的复合类型。
    - 指针的大小：在 64 位计算机中，指针占 8 个字节空间。
    - 指向常量对象的指针：常量指针



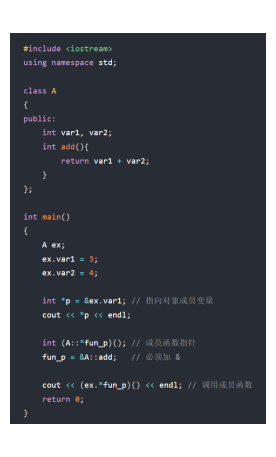
```
#include <iostream>
using namespace std;

int main(void)
{
    const int c_var = 10;
    const int * p = &c_var;
    cout << *p << endl;
    return 0;
}
```

- 指向函数的指针：函数指针



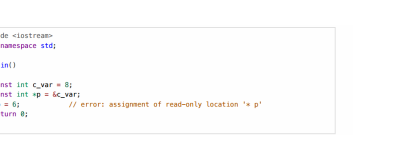
```
#include <iostream>
using namespace std;

int add(int a, int b){
    return a + b;
}

int main(void)
{
    int (*fun_p)(int, int);
    fun_p = add;
    cout << fun_p(1, 6) << endl;
    return 0;
}
```

- 指向对象成员的指针，包括指向对象成员函数的指针和指向对象成员变量的指针。特别注意：定义指向成员函数的指针时，要标明指针所属的类.



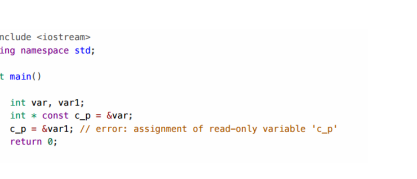
```
#include <iostream>
using namespace std;

class A
{
public:
    int var1, var2;
    int add(){
        return var1 + var2;
    }
};

int main()
{
    A ex;
    ex.var1 = 3;
    ex.var2 = 4;

    int *p = &ex.var1; // 指向对象成员变量
    cout << *p << endl;

    int (A::*fun_p)(); // 成员函数指针
    fun_p = &A::add;   // 必须加 &

    cout << (ex.*fun_p)() << endl; // 调用成员函数
    return 0;
}
```

- this 指针：指向类的当前对象的指针常量。
- 什么是野指针和悬空指针？
  - 悬空指针： 若指针指向一块内存空间，当这块内存空间被释放后，该指针依然指向这块a空间，此时，称该指针为“悬空指针”。
  - “野指针”是指不确定其指向的指针，未初始化的指针为“野指针”
- 指针和引用的区别是什么？
  - 指针所指向的内存空间在程序运行过程中可以改变，而引用所绑定的对象一旦绑定就不能改变。（是否可变）
  - 指针本身在内存中占有内存空间，引用相当于变量的别名，在内存中不占内存空间。（是否占内存）
  - 指针可以为空，但是引用必须绑定对象。（是否可为空）
  - 指针可以有多级，但是引用只能一级。（是否能为多级）
- 常量指针和指针常量的区别是什么？
  - 常量指针：常量指针本质上是个指针，只不过这个指针指向的对象是常量。
    - 特点：const 的位置在指针声明运算符 * 的左侧。只要 const 位于* 的左侧，无论它在类型名的左边或右边，都表示指向常量的指针。（可以这样理解，* 左侧表示指针指向的对象，该对象为常量，那么该指针为常量指针。）
    - 注意 1：指针指向的对象不能通过这个指针来修改，也就是说常量指针可以被赋值为变量的地址，之所以叫做常量指针，是限制了通过这个指针修改变量的值。



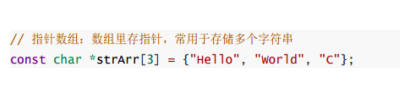
```
#include <iostream>
using namespace std;

int main()
{
    const int c_var = 8;
    const int *p = &c_var;
    *p = 6;            // error: assignment of read-only location '* p'
    return 0;
}
```

- 注意 2：虽然常量指针指向的对象不能变化，可是因为常量指针本身是一个变量，因此，可以被重新赋值。
- 指针常量的本质上是个常量，只不过这个常量的值是一个指针，和上面那个恰恰相反，不可以修改指针的指向，但可以通过对指针解引用修改指针指向的内容。
  - 特点：const 位于指针声明操作符右侧，表明该对象本身是一个常量， 左侧表示该指针指向的类型，即以 为分界线，其左侧表示指针指向的类型，右侧表示指针本身的性质。
  - 注意 1：指针常量的值是指针，这个值因为是常量，所以指针本身不能改变。



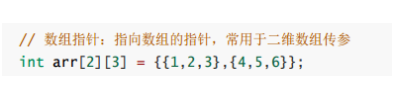
```
#include <iostream>
using namespace std;

int main()
{
    int var, var1;
    int * const c_p = &var;
    c_p = &var1; // error: assignment of read-only variable 'c_p'
    return 0;
}
```

- 注意 2：指针的内容可以改变。
- 函数指针和指针函数的区别是什么？
  - 指针函数本质是一个函数，只不过该函数的返回值是一个指针。
  - 函数指针本质是一个指针变量，只不过这个指针指向一个函数。
  - 两者区别
    - 函数指针是个指针，专门指向函数的指针变量，比如能存某个函数的地址，通过它能调用对应的函数。
    - 指针函数则是个函数，只是这个函数的返回值是指针类型，调用后会得到一个指针结果。
- 指针和引用值传递的区别是什么？
  - 值传递是指将参数的值复制一份，传递给函数或方法进行操作。在值传递中，函数或方法对参数进行修改不会影响到原始的变量值。
  - 指针引用是指将参数的内存地址传递给函数或方法，使得函数或方法可以直接访问和修改原始变量的值。在指针引用中，函数或方法对参数的修改会直接反映在原始变量上。
- 一级指针和二级指针的区别
  - 一级指针：存储普通变量的地址，通过 `*` 解引用访问变量的值。
  - 二级指针：存储一级指针的地址，通过 `**` 两次解引用访问最终变量的值。常用于函数中修改指针本身的值（如动态内存分配、链表节点操作）。
- 指针与数组
  - 指针与数组的区别？
    - 数组
      - 本质 — 一组连续内存空间的集合
      - 赋值 — 数组名是常量，不能被赋值
      - sizeof — 返回整个数组大小
      - 传参 — 退化为指针，丢失长度信息
    - 指针
      - 本质 — 存储地址的变量
      - 赋值 — 指针变量可以重新指向
      - sizeof — 返回指针本身大小（4/8字节）
      - 传参 — 本身就是指针
  - 指针数组和数组指针的区别？
    - 指针数组



```
// 指针数组：数组里存指针，常用于存储多个字符串
const char *strArr[3] = {"Hello", "World", "C"};
```

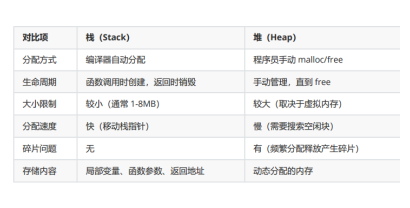
- 数组指针



```
// 数组指针：指向数组的指针，常用于二维数组传参
int arr[2][3] = {{1,2,3},{4,5,6}};
```

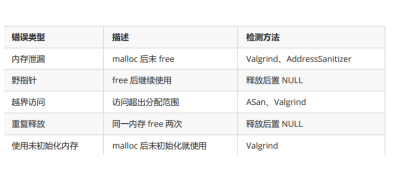
- 指针与字符串
  - 指针与字符串的关系？
    - 字符串在 C 语言中以 \0 结尾的字符数组表示。指针可以指向字符串常量或字符数组。

- 掌握内存管理
  - 栈和堆的区别？



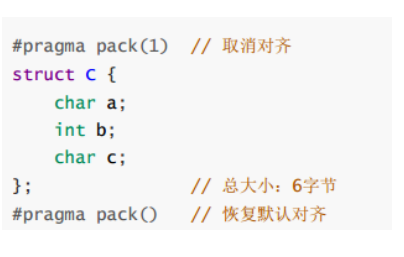
对比项	栈（Stack）	堆（Heap）
分配方式	编译器自动分配	程序员手动 malloc/free
生命周期	函数调用时创建，返回时销毁	手动管理，直到 free
大小限制	较小（通常 1-8MB）	较大（取决于虚拟内存）
分配速度	快（移动栈指针）	慢（需要搜索空闲块）
碎片问题	无	有（频繁分配释放产生碎片）
存储内容	局部变量、函数参数、返回地址	动态分配的内存

- malloc 和 free 的使用注意事项？
  - 核心原则：谁分配，谁释放；分配后检查 NULL；释放后置 NULL。
- 常见内存错误及检测工具



错误类型	描述	检测方法
内存泄漏	malloc 后未 free	Valgrind、AddressSanitizer
野指针	free 后继续使用	释放后置 NULL
越界访问	访问超出分配范围	ASan、Valgrind
重复释放	同一内存 free 两次	释放后置 NULL
使用未初始化内存	malloc 后未初始化就使用	Valgrind

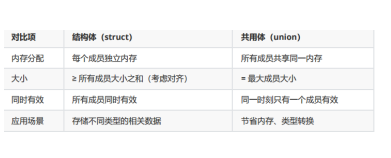
- 内存对齐是什么？为什么需要内存对齐？
  - 内存对齐规则
    - 1. 每个成员变量的起始地址必须是其大小的整数倍
    - 2. 结构体总大小必须是最大成员大小的整数倍
    - 3. 可以使用 #pragma pack(n) 修改对齐方式



```
#pragma pack(1)  // 取消对齐
struct C {
    char a;
    int b;
    char c;
};              // 总大小：6字节
#pragma pack()   // 恢复默认对齐
```

- 为什么需要对齐
  - 性能：CPU 按字长读取内存，对齐后一次读取即可，不对齐需要多次读取
  - 硬件限制：某些架构（如 ARM）要求严格对齐，否则触发异常

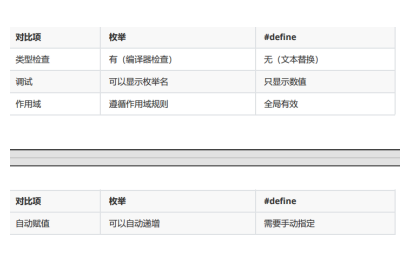
- 结构体和共用体
  - 结构体和共用体的区别？



对比项	结构体（struct）	共用体（union）
内存分配	每个成员独立内存	所有成员共享同一内存
大小	≥ 所有成员大小之和（考虑对齐）	= 最大成员大小
同时有效	所有成员同时有效	同一时刻只有一个成员有效
应用场景	存储不同类型的相关数据	节省内存、类型转换

- 联合体和结构体都是由若干个数据类型不同的数据成员组成。使用时，联合体只有一个有效的成员；而结构体所有的成员都有效。
- 对联合体的不同成员赋值，将会对覆盖其他成员的值，而对于结构体的对不同成员赋值时，相互不影响。
- 联合体的大小为其内部所有变量的最大值，按照最大类型的倍数进行分配大小；结构体分配内存的大小遵循内存对齐原则。
- C 和 C++ struct 的区别？
  - 在 C 语言中 struct 是用户自定义数据类型；在 C++ 中 struct 是抽象数据类型，支持成员函数的定义。
  - C 语言中 struct 没有访问权限的设置，是一些变量的集合体，不能定义成员函数；C++ 中 struct 可以和类一样，有访问权限，并可以定义成员函数。
  - C 语言中 struct 定义的自定义数据类型，在定义该类型的变量时，需要加上 struct 关键字，例如：struct A var;，定义 A 类型的变量；而 C++ 中，不用加该关键字，例如：A var;
- C++ 中 struct和Class区别是什么？
  - 默认访问权限不同：struct 默认访问权限为 public，class 默认访问权限为 private。
  - 默认继承方式不同：当使用继承时，两者的默认继承权限也不同，struct 默认继承方式为 public 继承，class 默认继承方式为 private 继承。

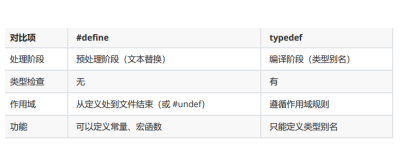
- 枚举的作用和使用场景
  - 枚举用于定义一组命名的整数常量，提高代码可读性
  - 枚举与宏定义的对比



对比项	枚举	#define
类型检查	有（编译器检查）	无（文本替换）
调试	可以显示枚举名	只显示数值
作用域	遵循作用域规则	全局有效

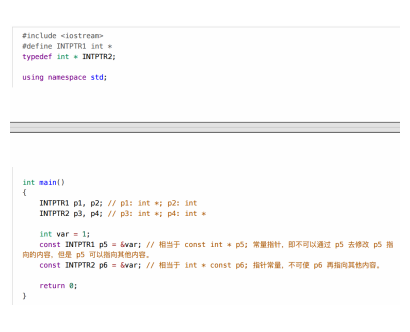
对比项	枚举	#define
自动赋值	可以自动递增	需要手动指定

- 宏定义和 typedef 的区别



对比项	#define	typedef
处理阶段	预处理阶段（文本替换）	编译阶段（类型别名）
类型检查	无	有
作用域	从定义处到文件结束（或 #undef）	遵循作用域规则
功能	可以定义常量、宏函数	只能定义类型别名

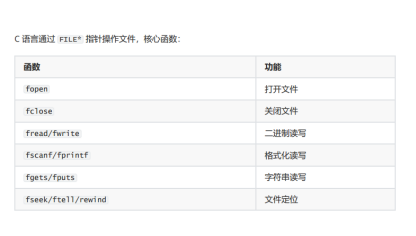
- 原理：#define 作为预处理指令，在编译预处理时进行替换操作，不作正确性检查，只有在编译已被展开的源程序时才会发现可能的错误并报错。typedef 是关键字，在编译时处理，有类型检查功能，用来给一个已经存在的类型一个别名，但不能在一个函数定义里面使用 typedef 。
- 功能：typedef 用来定义类型的别名，方便使用。#define 不仅可以为类型取别名，还可以定义常量、变量、编译开关等。
- 作用域：#define 没有作用域的限制，只要是之前预定义过的宏，在以后的程序中都可以使用，而 typedef 有自己的作用域。
- 指针的操作：typedef 和 #define 在处理指针时不完全一样



```
#include <iostream>
#define INTPTR1 int *
typedef int * INTPTR2;

using namespace std;
```

```
int main()
{
    INTPTR1 p1, p2; // p1: int *; p2: int
    INTPTR2 p3, p4; // p3: int *; p4: int *

    int var = 1;
    const INTPTR1 p5 = &var; // 相当于 const int * p5; 常量指针，即不可以通过 p5 去修改 p5 指向的内容，但是 p5 可以指向其他内容。
    const INTPTR2 p6 = &var; // 相当于 int * const p6; 指针常量，不可使 p6 再指向其他内容。

    return 0;
}
```

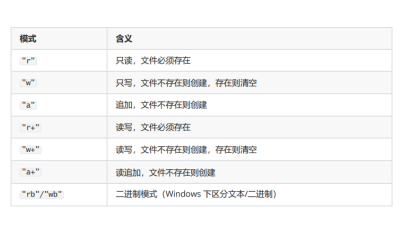
- 文件 I/O 的基本操作？



C 语言通过 `FILE*` 指针操作文件，核心函数：

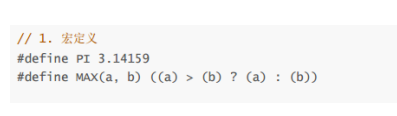
函数	功能
`fopen`	打开文件
`fclose`	关闭文件
`fread/fwrite`	二进制读写
`fscanf/fprintf`	格式化读写
`fgets/fputs`	字符串读写
`fseek/ftell/rewind`	文件定位

- 文件打开模式



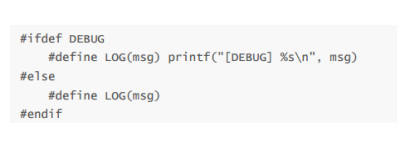
模式	含义
`"r"`	只读，文件必须存在
`"w"`	只写，文件不存在则创建，存在则清空
`"a"`	追加，文件不存在则创建
`"r+"`	读写，文件必须存在
`"w+"`	读写，文件不存在则创建，存在则清空
`"a+"`	读追加，文件不存在则创建
`"rb"/"wb"`	二进制模式（Windows 下区分文本/二进制）

- 预处理指令有哪些？
  - 宏定义



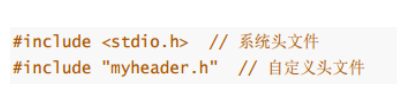
```
// 1. 宏定义
#define PI 3.14159
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

- 条件编译



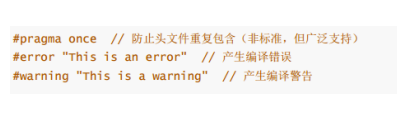
```
#ifdef DEBUG
    #define LOG(msg) printf("[DEBUG] %s\n", msg)
#else
    #define LOG(msg)
#endif
```

- 文件包含



```
#include <stdio.h>  // 系统头文件
#include "myheader.h"  // 自定义头文件
```

- 其他预处理指令



```
#pragma once  // 防止头文件重复包含（非标准，但广泛支持）
#error "This is an error"  // 产生编译错误
#warning "This is a warning"  // 产生编译警告
```

- 介绍const 作用及用法
  - 作用
    - const 修饰成员变量，定义成 const 常量，相较于宏常量，可进行类型检查，节省内存空间，提高了效率。
    - const 修饰函数参数，使得传递过来的函数参数的值不能改变
    - const 修饰成员函数，使得成员函数不能修改任何类型的成员变量（mutable 修饰的变量除外），也不能调用非 const 成员函数，因为非 const 成员函数可能会修改成员变量。
  - 类中的用法
    - const 成员变量
      - const 成员变量只能在类内声明、定义，在构造函数初始化列表中初始化。
      - const 成员变量只在某个对象的生存周期内是常量，对于整个类而言却是可变的，因为类可以创建多个对象，不同类的 const 成员变量的值是不同的。因此不能在类的声明中初始化 const 成员变量，类的对象还没有创建，编译器不知道他的值
    - const 成员函数
      - 不能修改成员变量的值，除非有 mutable 修饰；只能访问成员变量。
      - 不能调用非常量成员函数，以防修改成员变量的值
- define 和 const 的区别是什么
  - 编译阶段：define 是在编译预处理阶段进行替换，const 是在编译阶段确定其值。
  - 安全性：define 定义的宏常量没有数据类型，只是进行简单的替换，不会进行类型安全的检查；const 定义的常量是有类型的，是要进行判断的，可以避免一些低级的错误。
  - 内存占用：define 定义的宏常量，在程序中使用多少次就会进行多少次替换，内存中有多个备份，占用的是代码段的空间；const 定义的常量占用静态存储区的空间，程序运行过程中只有一份。
  - 调试：define 定义的宏常量不能调试，因为在预编译阶段就已经进行替换了；cons定义的常量可以进行调
    试。
  - const 的优点
    - 有数据类型，在定义式可进行安全性检查
    - 可调试
    - 占用较少的空间
- 大端和小端的区别？
  - 大端：高字节存储在低地址
  - 小端：低字节存储在低地址

C++

- C++ static作用是什么
  - static 定义静态变量，静态函数.保持变量内容持久：static 作用于局部变量，改变了局部变量的生存周期，使得该变量存在于定义后直到程序运行结束的这段时间。
  - 隐藏：static作用于全局变量和函数，改变了全局变量和函数的作用域，使得全局变量和函数只能在定义它的文件中使用，在源文件中不具有全局可见性。（注：普通全局变量和函数具有全局可见性，即其他的源文件也可以使用。）
  - static 作用于类的成员变量和类的成员函数，使得类变量或者类成员函数和类有关，也就是说可以不定义类的对象就可以通过类访问这些静态成员。注意：类的静态成员函数中只能访问静态成员变量或者静态成员函数，不能将静态成员函数定义成虚函数。
- static 在类中使用的注意事项有哪些
  - 静态成员变量是在类内进行声明，在类外进行定义和初始化，在类外进行定义和初始化的时候不要出现static关键字和private、public、protected 访问规则。
  - 静态成员变量相当于类域中的全局变量，被类的所有对象所共享，包括派生类的对象。
  - 静态成员变量可以作为成员函数的参数，而普通成员变量不可以。
  - 静态数据成员的类型可以是所属类的类型，而普通数据成员的类型只能是该类类型的指针或引用。
  - static 静态成员函数
    - 静态成员函数不能调用非静态成员变量或者非静态成员函数，因为静态成员函数没有 this 指针。静态成员函数做为类作用域的全局函数。
    - 静态成员函数不能声明成虚函数（virtual）、const 函数和 volatile 函数。
- static 全局变量和普通全局变量的异同是什么
  - 相同点
    - 存储方式：普通全局变量和 static 全局变量都是静态存储方式。
  - 不同点
    - 作用域：普通全局变量的作用域是整个源程序，当一个源程序由多个源文件组成时，普通全局变量在各个源文件中都是有效的；静态全局变量则限制了其作用域，即只在定义该变量的源文件内有效，在同一源程序的其它源文件中不能使用它。由于静态全局变量的作用域限于一个源文件内，只能为该源文件内的函数公用，因此可以避免在其他源文件中引起错误。
    - 初始化：静态全局变量只初始化一次，防止在其他文件中使用。
- C++ 静态变量的使用场景是什么？未初始化的全局静态变量呢？
  - 全局静态变量（static 修饰的全局变量）
    - 作用：限制变量仅在当前文件内可见（避免不同文件中同名变量冲突），但生命周期是整个程序运行期间。
    - 场景：当多个文件需要独立使用同名变量（如统计各模块的内部计数），但不希望被其他文件访问或修改时。例：static int count = 0;（仅当前 .cpp 文件可访问，其他文件即使声明 extern int count 也无法使用）。
  - 局部静态变量（函数内的 static 变量）
    - 作用：变量在函数第一次调用时初始化，后续调用不再重新初始化，值会被保留（生命周期全局，作用域仅限函数内）。
    - 场景：记录函数被调用的次数（如 static int call_count = 0; call_count++;）；单例模式中，确保全局只存在一个实例（如函数内返回静态对象的指针）；避免频繁创建销毁临时对象（如工具函数中复用的缓冲区）。
  - 类静态成员变量（static 修饰的类成员）
    - 作用：属于整个类而非某个对象，所有对象共享该变量，生命周期全局，需在类外单独初始化。
    - 场景：统计类的实例数量（如 static int total;，在构造函数中 total++，析构函数中 total--）；存储类级别的常量或共享配置（如 static const int MAX_SIZE = 100;）。
  - 未初始化的全局静态变量
    - 自动初始化：编译器会将其默认初始化为 0（包括数值类型为 0，指针类型为 nullptr 等）。这是因为全局静态变量存放在内存的 BSS 段（未初始化数据段），程序启动时系统会自动将该段所有数据清零。
    - 作用域限制：和初始化的全局静态变量一样，仅在当前文件内可见，不影响其他文件的同名变量。

分支主题 3

分支主题 4

Presented with xmind